Ca' Foscari University of Venice

Development Methodology

Matteo Zanioli

matteo.zanioli@unive.it

Complex architecture

- **Complex architectures** are system designs characterized by a high number of interconnected components, often distributed, built to address advanced requirements for scalability, performance, resilience, and flexibility. These architectures are frequently modular and can include elements such as microservices, geographic distribution, load balancing, distributed caching, and real-time data and state management.

Primary dimensions of complexity

- *Components and Connectivity*: The number of modules and subsystems and the way they interact (e.g., microservices vs. monolithic architectures).
- *Distribution and Coordination*: Systems distributed across multiple locations or cloud providers, with mechanisms to ensure proper synchronization and consistency.
- *Scalability and Flexibility Requirements*: Handling increased loads and the need for rapid changes or expansion, such as adding new services or scaling horizontally.
- *State and Data Management*: Managing data integrity, session states, and synchronization in distributed settings.

Challenges Associated with Complex Architectures

- *Increased Maintenance and Operational Overhead*: More components to manage, monitor, and update.
- *Higher Security and Compliance Demands*: More entry points and interfaces, each adding potential security vulnerabilities.
- *Performance Considerations*: Complex architectures can introduce latency due to inter-service communication, requiring advanced load balancing and caching strategies.
- *Debugging and Monitoring Difficulties*: Identifying issues across distributed services or components can be harder and often requires sophisticated observability tools.

When is Complexity Justified?

- *High Scalability Requirements*: When the system needs to handle millions of concurrent users or transactions, a microservices or distributed approach may be necessary.
- *Rapid Deployment and Flexibility*: If there's a need for frequent updates, modular architectures (like microservices) allow independent deployment.
- *Failover and Fault Tolerance*: Complex architectures often include redundancy and failover mechanisms to ensure high availability and disaster recovery.

- Examples
  - E-commerce Platforms: Need for handling spikes in traffic and separating different business functions (e.g., payments, product catalog).
  - Social Media or Streaming Platforms: High throughput demands, along with real-time data and session management for millions of users.
  - Financial Systems: Require strong data consistency, security, and compliance, often involving multiple interdependent services.
- Considerations Before Adopting Complexity
  - *Cost vs. Benefit Analysis*: Determining if the operational costs and additional skill requirements are offset by the benefits.
  - *Team Expertise and Resources*: A highly skilled team is often necessary to manage complex systems effectively.
  - *Anticipated Growth and Evolution*: It may be wise to start with a simpler architecture and transition to a complex one as needs grow.

Scalability

- Scalability is the capability of a system, network, or process to **handle a growing amount of work** or its **potential to accommodate growth**.
- It refers to the system's ability to maintain or improve its performance and efficiency as it is **scaled up** (by increasing the resources, such as CPU, memory, or storage) or **scaled out** (by adding more machines or nodes).
- Scalable systems are designed to support increasing workloads without compromising responsiveness or stability, making them crucial for applications expected to handle higher user demands or data loads over time.

Scalability

Scaling up (vertical scalability)

- Involves adding more power (CPU, RAM, storage) to an existing machine.
- *Simplicity*: Easier to implement, as it doesn't require distributing data or traffic across multiple nodes.
- *Consistency*: Avoids issues related to data synchronization, session management, or load balancing.
- Limited by *hardware constraints*: There's a maximum capacity a single machine can handle.
- *Increased single-point failure risk*: Since all operations depend on a single instance, it's more vulnerable if the system goes down.
- Use cases: Often used for databases or applications with consistent but intensive workloads, where a single powerful server is efficient.

Scaling out (horizontal scalability)

- Involves adding more machines or nodes to the system to distribute the workload.
- *High fault tolerance*: Reduces single-point failures by distributing tasks across multiple nodes.
- *Flexibility and Unlimited Growth Potential*: New nodes can be added to support increasing loads, theoretically without a limit.
- *Complexity*: Requires load balancing, data distribution (e.g., sharding), and session management across nodes.
- *Data Consistency*: Managing state and data integrity across distributed nodes can be challenging.
- Use Cases: Preferred for web applications, distributed databases, and systems with highly variable or unpredictable traffic.

Scalability

Cloud

- *Flexibility*: Cloud environments allow easy vertical and horizontal scaling, often automated based on demand.
- *Cost Efficiency*: Cloud providers offer pay-as-you-go pricing models, making it more cost-effective to scale resources dynamically.
- *Elasticity*: Cloud providers like AWS, Azure, and Google Cloud provide automatic scaling services, allowing systems to adapt instantly to load changes.
- *Limitations*: Potential data sovereignty and security issues due to data being stored on third-party servers; dependence on network stability.
- *Use Cases*: Suitable for applications with unpredictable demand, rapid scaling needs, or multi-region support.

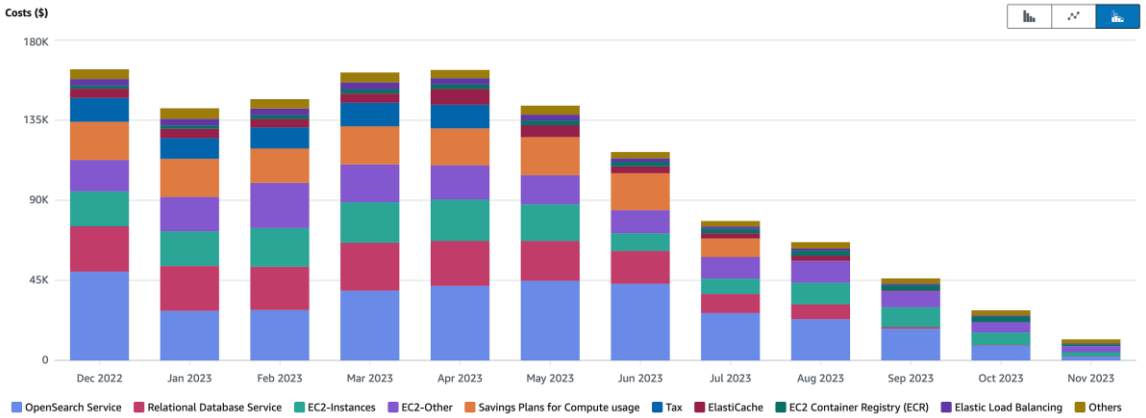
On premise

- *Control*: Complete control over hardware and software, with reduced dependence on external vendors.
- *Data Security*: Enhanced security and compliance options, as data remains within the organization.
- *Limitations*: Scaling vertically on-premise requires significant hardware investments, and horizontal scaling requires setting up additional physical servers.
- *Use Cases*: Common in industries with strict data regulations or for applications with predictable demand and budgets.

Ca' Foscari University of Venice

About on prem vs cloud

- The big cloud exit FAQ - https://world.hey.com/dhh/the-big-cloud-exit-faq-20274010 - written by David Heinemeier Hansson
- $3,2M cloud budget in 2022
- Estimation: save $7M by 2027



Scalability strategy

Load balancing

- *Description*: Distributes incoming network or application traffic across multiple servers to ensure no single server becomes overloaded.
- *Benefits*:
  - Enhances system reliability and responsiveness by evenly distributing traffic.
  - Helps in achieving horizontal scaling by routing requests to various nodes.
- *Example*: A web application deployed on multiple servers with a load balancer to distribute user requests. Amazon's Elastic Load Balancing (ELB) is a cloud-based example that dynamically routes traffic.

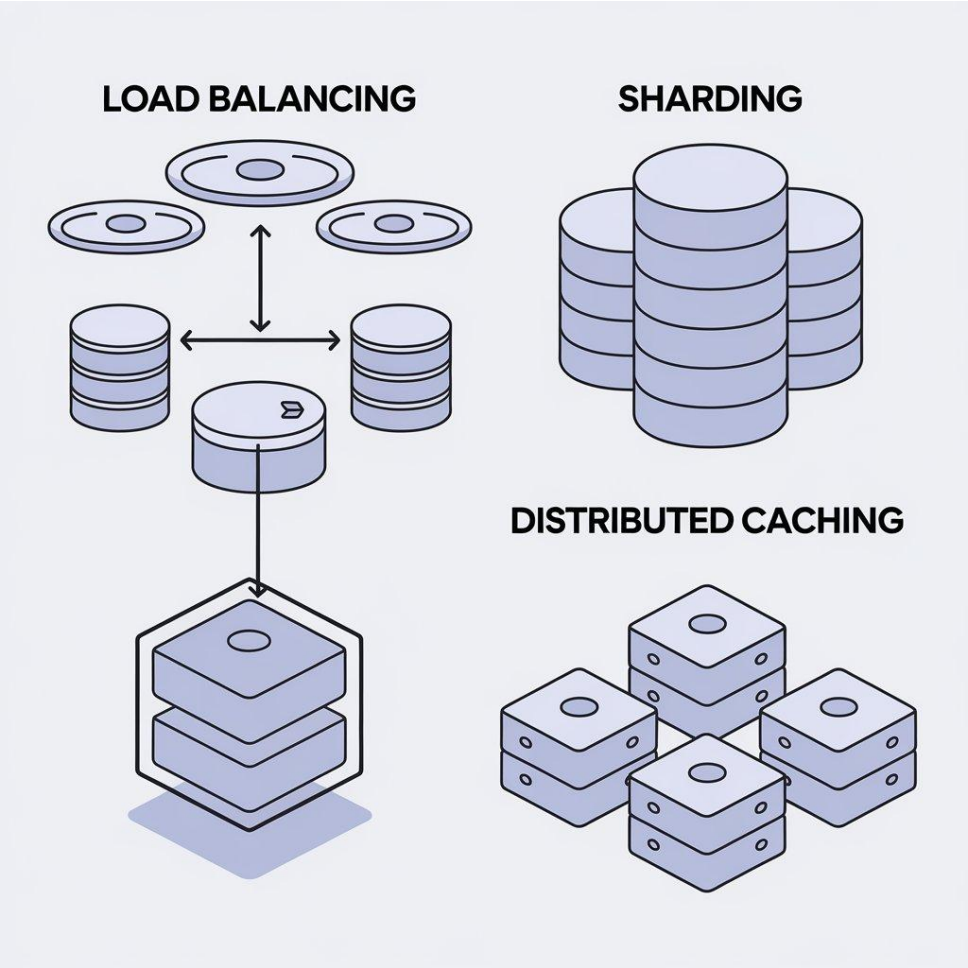
Sharding

- *Description*: Splits large datasets into smaller, more manageable segments, or "shards," typically distributed across multiple database nodes.
- *Benefits*:
  - Reduces load on individual nodes by allowing each to handle only a portion of the data.
  - Enables horizontal scaling of databases, critical for large-scale applications.
- *Example*: A social media platform might shard user data by region, so user data from Europe is stored separately from data in North America, improving access speed and reducing database bottlenecks.

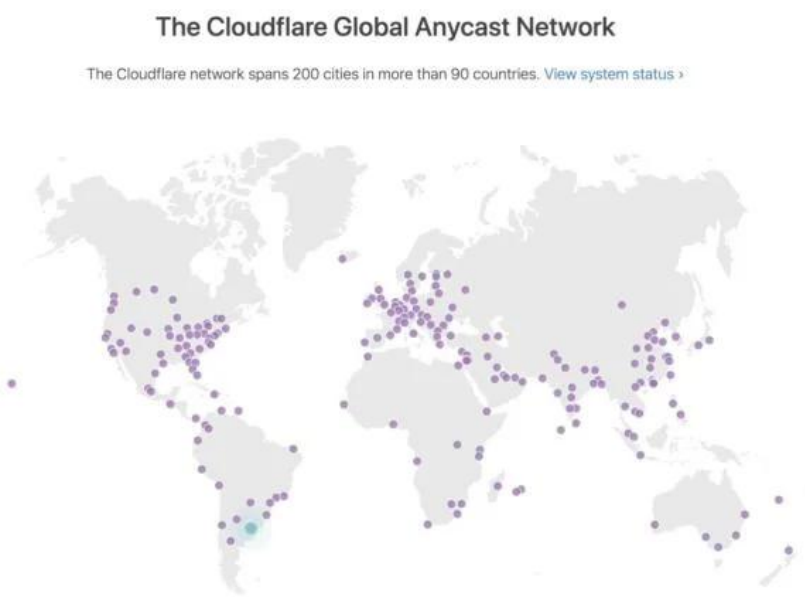
Scalability strategy

Distributed Caching

- *Description*: Stores frequently accessed data in a temporary storage layer across multiple servers to reduce database load and speed up response times.
- *Benefits*:
  - Reduces database requests, enhancing performance and scalability.
  - Allows data to be accessed faster, improving user experience for high-traffic applications.
- *Example*: Content Delivery Networks (CDNs) like Cloudflare use distributed caching to serve static content from nodes geographically closer to users, reducing latency.



CDN examples



- Akamai operates one of the most extensive networks in the world, with over 240,000 edge servers located in more than 130 countries. These servers are strategically placed near end-users to reduce latency and enhance performance.
- Cloudflare runs a global network with edge servers in over 200 cities across more than 100 countries. This wide distribution enables Cloudflare to deliver content with low latency to users worldwide.These global networks allow Akamai and Cloudflare to provide efficient CDN services, improving the end-user experience through fast and reliable content delivery.

Serverless

- Serverless computing (or simply "serverless") is a cloud computing execution model.
- No server management
  - developers don't need to manage servers
  - the cloud provider automatically handles infrastructure
  - resources are allocated dynamically
- Pay-per-use pricing
  - you only pay for actual compute time used
  - no charges when code isn't running
  - typically billed in millisecond increments
- Autoscaling
  - automatic scaling based on demand
  - scales from zero to peak loads instantly
  - no manual intervention required
- Event-driven
  - functions are triggered by events
  - common triggers include
    - HTTP requests
    - Database changes
    - File uploads
    - Queue messages
    - Scheduled tasks

Serverless

- Benefits
  - Reduced operational costs
  - No infrastructure management
  - Improved developer productivity
  - Built-in high availability
  - Automatic scaling
- Common Use Cases:
  - API backends
  - Data processing
  - Real-time file processing
  - Scheduled tasks
  - IoT backends
  - Chatbots
- Popular Serverless Platforms
  - AWS Lambda
  - Azure Functions
  - Google Cloud Functions
  - IBM Cloud Functions
  - CloudFlare Workers
- Limitation
  - Cold start latency
  - Limited execution duration
  - Vendor lock-in potential
  - Limited local testing
  - Complex debugging

Serverless – interesting products

OpenWhisk

- https://openwhisk.apache.org/
- Open Source, distributed Serverless Cloud Platform that executes functions in response to events at any scale.
- OpenWhisk manages the infrastructure, servers and scaling using Docker containers so you can focus on building applications.

Nuvolaris

- https://www.nuvolaris.io/
- "we make Kubernetes a serverless solution".
- designed to simplify cloud-native application development across various Kubernetes environments.
- Portability: Deploy applications seamlessly across multiple cloud providers without code modifications.
- Simplicity: Streamline the deployment process, eliminating the complexities associated with Kubernetes management.

Performance

- In computing, **performance** refers to how efficiently and effectively a system, application, or component completes tasks under specified conditions. Key metrics that define performance include **response time** (how quickly the system processes requests), **throughput** (the amount of work handled in a given timeframe), and **resource utilization** (efficient use of CPU, memory, and storage). High performance is often characterized by low **latency**, high **reliability**, and the system's **ability to scale** under increasing loads without degradation, ensuring smooth user experience and responsiveness.

Performance

Latency

- The delay before data begins to transfer after a request is made. Minimizing latency is critical for real-time applications like gaming or video streaming.
- Factors Affecting Latency:
  - Network Distance: Data often travels across multiple networks, each adding a small delay (network hops).
  - Processing Delays: Backend processing or querying complex databases can add latency.
  - Server Load: High traffic can increase latency if servers struggle to keep up.
- Example: In online gaming, high latency causes lag, disrupting the user experience. Techniques to reduce latency include reducing network hops and optimizing backend processing times.

Throughput

- Throughput is the total volume of data processed over a specific period, typically measured in transactions per second (TPS) or requests per second (RPS).
- High throughput is essential for applications handling a large number of simultaneous requests, ensuring that the system meets demand without slowing down.
- Factors Affecting Throughput:
  - System Architecture: Distributed systems or microservices can increase throughput by processing multiple requests concurrently.
  - Database Performance: Optimized databases and the use of read replicas help systems handle more data in less time.
  - Hardware Capacity: Sufficient memory and CPU resources are needed to avoid bottlenecks.
- Example: E-commerce sites like Amazon monitor throughput to ensure they can process thousands of transactions per second during peak times like Black Friday.

Performance

Response time

- Response time is the total time taken for a system to process a user request and return a result. Low response times improve the user experience, especially for applications requiring immediate feedback.
- Factors Affecting Response Time:
  - Backend Processing Time: Complex calculations or large data transfers can slow response times.
  - I/O Operations: Slow database or file storage access can increase response time.
  - Concurrency Management: Systems that efficiently handle concurrent requests tend to have better response times.
- Search engines like Google prioritize response time; if queries aren't returned within milliseconds, users may leave. Techniques include query optimization and caching frequently accessed data.

Error rate

- Error rate is the percentage of failed requests out of the total requests over a given period. A high error rate often signals issues within the system, like configuration errors, code bugs, or server overload.
- Monitoring error rates is crucial to maintain reliability. A consistently low error rate indicates that users can rely on the application to function without interruptions.
- Factors Affecting Error Rate:
  - System Load: High traffic can cause bottlenecks, leading to failed requests if servers cannot keep up.
  - Software Bugs: Errors in code, misconfigurations, or poor exception handling can increase error rates.
  - Dependency Failures: External services or APIs that a system relies on can also impact error rate.
- Example: Payment platforms like PayPal monitor error rates meticulously, as high error rates could lead to transaction failures

Performance - tools

- Application Performance Monitoring (APM):
  - Tools like New Relic, AppDynamics, and Datadog allow tracking and analyzing application performance.
  - Example: APMs track database query times, CPU usage, and error rates in real-time. A company might use New Relic to monitor API performance, receiving alerts when response times exceed acceptable thresholds.
- Logging:
  - Implement structured logging for error detection and bottleneck identification. Logs provide essential context, especially in distributed systems where tracing user requests across services is challenging.
  - Example: Microservice architectures often use ELK stack (Elasticsearch, Logstash, Kibana) for centralized logging, making it easier to detect slow-performing services.
- Distributed Tracing:
  - Tools like Jaeger and Zipkin track requests across services, helping diagnose where bottlenecks occur in distributed systems.
  - Example: A company using microservices for an e-commerce platform can trace a purchase transaction across services (catalog, cart, payment) to see where delays occur.

Ca' Foscari University of Venice

Optimization Patterns and Strategies

- Command Query Responsibility Segregation (CQRS):
  - Separates read and write operations to optimize performance and scalability.
  - Example: An e-commerce platform could use CQRS by storing customer order history in a read-only database optimized for search queries, while order processing is handled by a write-optimized database.
- Microservices for Optimized Response Times:
  - Microservices architecture splits an application into smaller, loosely-coupled services.
  - Benefits include isolated scaling, better fault tolerance, and faster deployment.
  - Example: A travel booking platform with separate microservices for flights, hotels, and cars can scale each service independently. During peak flight booking seasons, the flight service can be scaled up without affecting other services.
- Event-Driven Architecture:
  - Uses event messaging to decouple services, reducing processing time.
  - Example: When a user purchases an item on an e-commerce site, the order service triggers events for inventory, shipping, and notifications, enabling asynchronous processing and faster response times.
- Bulkhead Pattern
  - This pattern isolates different parts of a system to prevent failures in one component from cascading to others, much like compartments (bulkheads) on a ship that contain water in the event of a hull breach.
  - It limits the impact of failures, helping to maintain partial functionality even if one part of the system encounters issues. This can improve the overall reliability and resilience of the application.
  - Example: In a microservices architecture, an e-commerce application might separate inventory, payment, and order-processing services into different "bulkheads." If the inventory service fails, it won't affect the payment service, allowing users to proceed with payments for other products.
- Cache-aside Pattern (Lazy Loading)
  - With cache-aside (or lazy loading), data is loaded into the cache only when requested. If the data isn't in the cache, it's retrieved from the database and then stored in the cache for future access.
  - Reduces load on the database by caching frequently requested data. Improves response times for users accessing the cached data while keeping cache storage efficient.
  - Example: A news website could use cache-aside to store articles in the cache only after they're requested. When a user accesses an article for the first time, the system fetches it from the database and caches it. Subsequent requests for the same article are served directly from the cache, speeding up load times.

Caching

- In computing, **caching** is a technique used to temporarily **store frequently accessed data** in a **faster storage layer**, or cache, so that future requests for that data can be served more quickly. Instead of repeatedly querying the primary data source (like a database or remote server), **caching enables the system to retrieve data from a local or in-memory cache**, significantly reducing response times and resource usage. Caching is widely applied in web applications, databases, and content delivery networks (CDNs) to improve performance, reduce latency, and handle higher loads efficiently.

- Type of caching
  - Client-side caching: Stores data on the client's browser.
  - Server-side caching: Uses tools like Redis or Memcached to store frequently accessed data.
- Content Delivery Networks (CDNs):
  - CDNs like Cloudflare and Akamai distribute cached content across global servers, allowing users to access data from servers nearest to them.
  - Example: Websites with global traffic rely on CDNs for fast delivery of static resources like images, CSS, and JavaScript, reducing load times for international users.

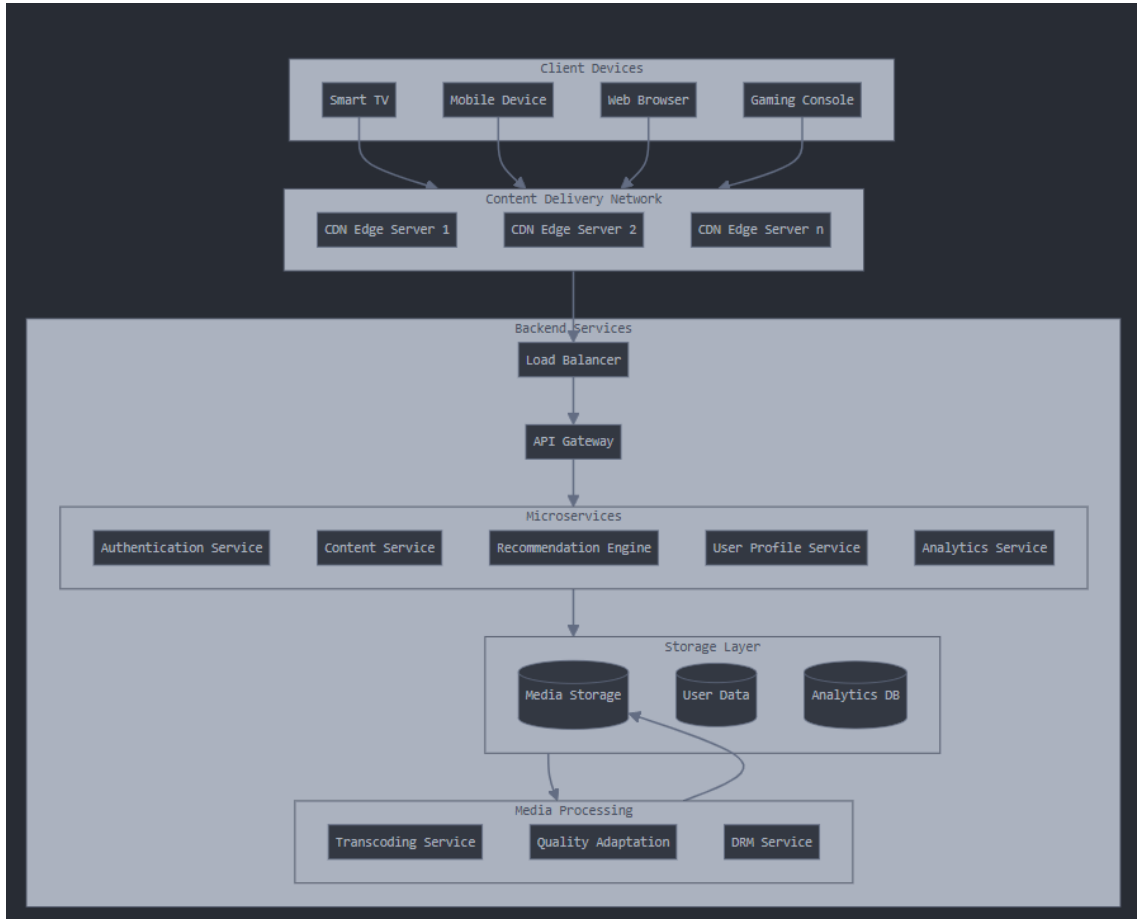
Caching - examples

- Web application
  - Database cache
  - Server cache
  - CDN
  - Browser cache
  - OS-level cache
- Video streaming services
  - Local device cache
  - Edge server cache
  - CDN regional cache
- Operating System
  - hierarchical caching structure (CPU L1, L2, and L3), RAM cache, and disk cache

Ca' Foscari University of Venice

Video streaming architecture

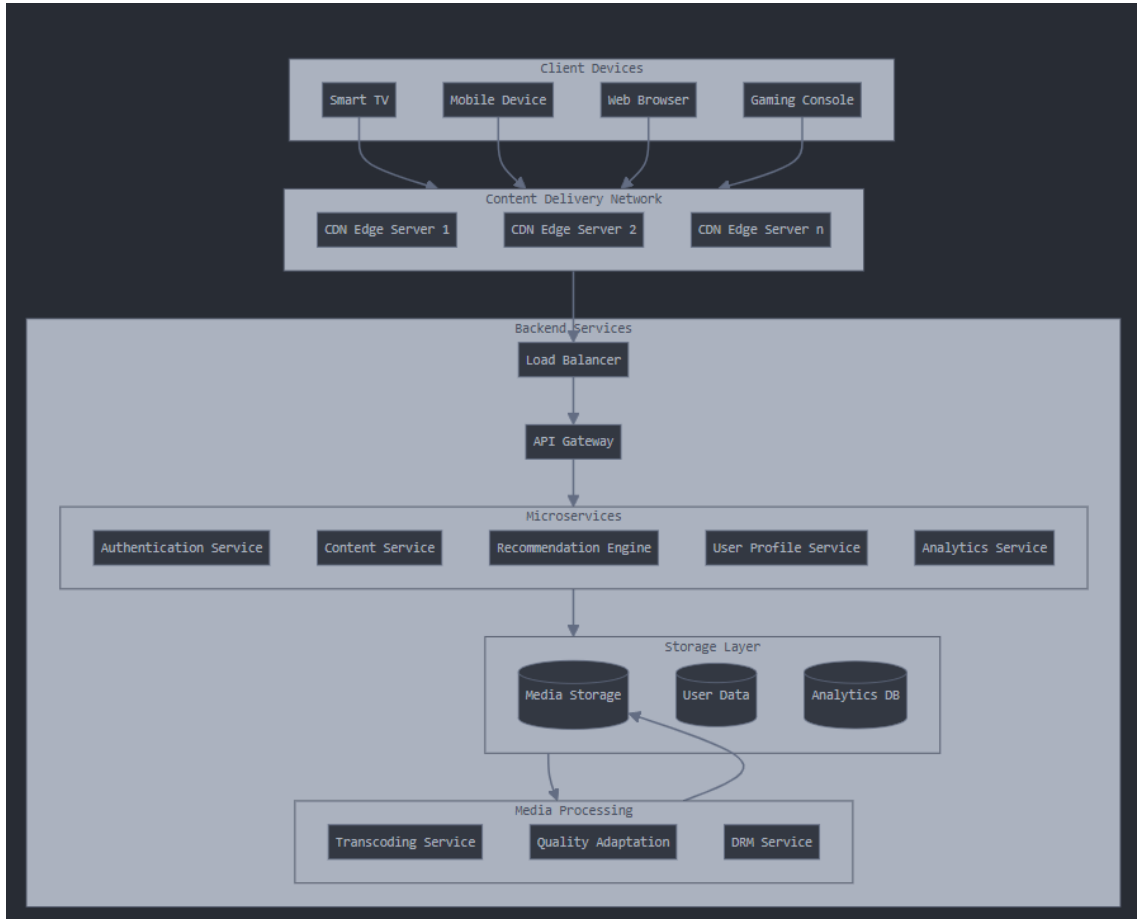
- CDN
  - Thousands of server nodes distributed globally
  - Stores cached copies of content close to end-users
  - Reduces latency and bandwidth costs
  - Handles about 95% of streaming traffic
- Backend services
  - Load Balancer distributes traffic across services
  - API Gateway handles client requests and routing
  - Microservices contains the business logic
- Storage layer
  - Media Storage: Original and transcoded content
  - User Data: Preferences, viewing history, accounts
  - Analytics Database: Viewing patterns, performance metrics
- Media processing
  - Transcoding Service: Creates multiple quality versions of videos
  - Quality Adaptation: Manage different bitrates and resolutions
  - DRM Service: Handles content protection



Ca' Foscari University of Venice

NETFLIX

- Uses AWS for most of their cloud infrastructure
- Implements chaos engineering (Chaos Monkey) for resilience testing
- Uses Multiple availability zones for redundancy
- Employs sophisticated caching strategies at various levels
- Implements their own adaptive streaming algorithms
- Uses multiple quality encodings for each video (typically 120+ versions)
- Technologies used
  - Java and Node.js for microservices
  - Cassandra for distributed databases
  - Redis for caching
  - Kafka for message queuing
  - ELK Stack for logging
- Bibliograpy:
  - https://netflixtechblog.com/
  - https://netflix.github.io/
  - https://netflixtechblog.com/netflix-presentation-videos-from-aws-re-invent-2013-807532f84f90



Ca' Foscari University of Venice

Redis – real time data platform

We're the world's fastest in-memory database. From our open source origins in 2011 to becoming the #1 cited brand for caching solutions, we've helped more than 10,000 customers build, scale, and deploy the apps our world runs on. With cloud and on-prem databases for caching, vector search, and more, we're helping digital businesses set a new standard for speed.

We believe

Fast matters the second you start

We get you working immediately, so you can spend more time building and less time on hefty data management or incompatible tech stacks and languages.

Fast apps are nonnegotiable

We bring you real-time data that meets your users' demands and puts an end to latency, slow data processing, and over-utilization costs as you know it.

Fast means no interruptions

We have hundreds of developers, demos, and docs that help you work how you want and get it done right—whether you're stuck or building your next big thing.

Redis Products **Solutions** Support Company Docs Pricing

USE CASES

- Caching
- Deduplication
- Fast data ingest
- Feature stores
- Session management
- Vector database

INDUSTRIES

- Financial services
- Gaming
- Healthcare
- Retail

CUSTOMER CASE STUDIES

Read stories

NIANTIC

Optimizing Pokémon GO with a Redis cluster

See more

Ca' Foscari University of Venice

redis-demo.py

```python
# 1. Strings - storing user preferences
print("1. Setting user preferences using STRING:")
redis_client.set('user:123:language', 'english')
print(f"User language preference: {redis_client.get('user:123:language')}\n")

# 2. Lists - managing a shopping cart
print("2. Managing shopping cart using LIST:")
cart_key = 'cart:user:123'
redis_client.rpush(cart_key, 'laptop')
redis_client.rpush(cart_key, 'mouse')
redis_client.rpush(cart_key, 'keyboard')
print(f"Cart items: {redis_client.lrange(cart_key, 0, -1)}")
print(f"Cart size: {redis_client.llen(cart_key)}\n")

# 3. Sets - product categories
print("3. Managing product categories using SET:")
electronics_key = 'category:electronics'
redis_client.sadd(electronics_key, 'laptop', 'phone', 'tablet', 'mouse')
print(f"Electronics products: {redis_client.smembers(electronics_key)}")
print(f"Is 'laptop' an electronic? {redis_client.sismember(electronics_key, 'laptop')}\n")

# 4. Hashes - product details
print("4. Storing product details using HASH:")
product_key = 'product:laptop'
redis_client.hset(product_key, mapping={
    'name': 'Gaming Laptop',
    'price': '999.99',
    'stock': '5'
})
print(f"Product details: {redis_client.hgetall(product_key)}\n")

# 5. Sorted Sets - product rankings
print("5. Product rankings using SORTED SET:")
rankings_key = 'products:viewed'
redis_client.zadd(rankings_key, {'laptop': 100, 'mouse': 50, 'keyboard': 75})
print("Most viewed products:")
for product, views in redis_client.zrevrange(rankings_key, 0, -1, withscores=True):
    print(f"- {product}: {int(views)} views")
print()

# 6. Expiration - temporary discount code
print("6. Setting expiring keys for temporary data:")
redis_client.set('discount:summer2024', 'SUMMER20', ex=3600)  # expires in 1 hour
print(f"Discount code: {redis_client.get('discount:summer2024')}")
print(f"Time until expiration: {redis_client.ttl('discount:summer2024')} seconds\n")
```

```
mzanioli@Erdos:~/unive/devmet24/myrepos/DevMet24-redis$ python3 redis-demo.py
=== Redis Shopping Cart Demo ===

1. Setting user preferences using STRING:
User language preference: english

2. Managing shopping cart using LIST:
Cart items: ['laptop', 'mouse', 'keyboard']
Cart size: 3

3. Managing product categories using SET:
Electronics products: {'tablet', 'mouse', 'phone', 'laptop'}
Is 'laptop' an electronic? 1

4. Storing product details using HASH:
Product details: {'name': 'Gaming Laptop', 'price': '999.99', 'stock': '5'}

5. Product rankings using SORTED SET:
Most viewed products:
- laptop: 100 views
- keyboard: 75 views
- mouse: 50 views

6. Setting expiring keys for temporary data:
Discount code: SUMMER20
Time until expiration: 3600 seconds
```

Ca' Foscari University of Venice

NETFLIX

- Netflix utilizes Redis to enhance user session management and improve the streaming experience.
- Handling millions of concurrent users, Netflix manages data such as login information, viewing history, and personal preferences.
- Redis, with its in-memory data storage and rapid retrieval capabilities, enables efficient management of this information.
- Through Redis, Netflix delivers uninterrupted streaming and a highly personalized experience, ensuring rapid response times and global scalability.

- How Netflix employs Redis:
  - Session Management: Redis stores user session data, allowing Netflix to access session information in real-time, supporting millions of simultaneous requests with minimal latency.
  - Personalized Recommendations: By utilizing Redis, Netflix manages personalized recommendations. When a user watches content, Redis stores information such as viewing time and preferences, enabling immediate suggestions upon the next platform access.
  - Microservices Caching: Netflix's microservices architecture leverages Redis as a caching layer, allowing various microservices to quickly access shared data and reducing the need for constant queries to primary databases.
  - Rate Limiting: Redis facilitates rate limiting, controlling the number of requests a user can make within a specific timeframe. This helps maintain platform performance stability during traffic surges.

Sources:
- https://github.com/Netflix/nf-data-explorer
- https://dev.to/slk5611/redis-queue-optimising-workflows-with-message-queues-2341
- https://www.infoq.com/news/2022/10/netflix-timestone-priority-queue/

Redis with docker

- Requirement:
  - `sudo apt install docker-compose`
- Command to execute:
  - `docker-compose up --build`

docker-compose

- Docker Compose is a tool that helps you define and run multi-container Docker applications.
- Service Definition: Define all your application's services, networks, and volumes
- Environment Management: Handle multiple environments (development, production, testing)
- Container Orchestration: Manage multiple containers as a single application
- Designed for single-host deployments
  - for multi-host production deployments you might want consider solution like K8s or Docker Swarm

- Common commands
  - `docker-compose up #start service`
  - `docker-compose up -d #detached mode`
  - `docker-compose down #stop services`
  - `docker-compose logs #view logs`
  - `docker-compose ps #list running containers`
  - `docker-compose build #build a service`
- Common use:
  - Local development environments
  - Automated testing environments
  - Single host deployments
  - CI/CD workflows

Redis with docker

```yaml
# docker-compose.yml
version: '3.8'
services:
  redis:
    image: redis:latest
    ports:
      - "6379:6379"
    volumes:
      - redis_data:/data
    command: redis-server --appendonly yes

  redis-app:
    build: .
    depends_on:
      - redis
    environment:
      - REDIS_HOST=redis
      - REDIS_PORT=6379

volumes:
  redis_data:
```

- configuration file of docker-compose
- two services (redis and redis-app)
- redis
  - only redis server
  - data persistence
- redis-app
  - python 3.9 (specified in Dockerfile)
  - our application (app.py)
  - library redis.py installed through pip
  - needed dependencies

Redis with docker

- Dockerfile
- install redis
- copy python app

```
FROM python:3.9-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install -r requirements.txt
COPY app.py .
CMD ["python", "app.py"]
```

Ca' Foscari University of Venice

Redis with docker

```python
import redis
import os
import time

def main():
    # Redis connection
    redis_client = redis.Redis(
        host=os.getenv('REDIS_HOST', 'localhost'),
        port=int(os.getenv('REDIS_PORT', 6379)),
        decode_responses=True
    )

    # Example 1: Basic operations with strings
    print("=== Example 1: Basic operations ===")
    redis_client.set('course', 'Redis with Docker')
    value = redis_client.get('course')
    print(f"Retrieved value: {value}")

    # Example 2: Working with lists
    print("\n=== Example 2: Lists ===")
    redis_client.rpush('students', 'Mark', 'Anna', 'Louis')
    students = redis_client.lrange('students', 0, -1)
    print(f"Students list: {students}")

    # Example 3: Using hashes
    print("\n=== Example 3: Hashes ===")
    redis_client.hset('user:1', mapping={
        'name': 'John',
        'surname': 'Smith',
        'age': '25'
    })
    user = redis_client.hgetall('user:1')
    print(f"User data: {user}")

    # Example 4: Key expiration
    print("\n=== Example 4: Key expiration ===")
    redis_client.setex('session', 10, 'active')
    print("Key 'session' set with 10 seconds expiration")
    print(f"Current value: {redis_client.get('session')}")
    time.sleep(11)
    print(f"Value after 11 seconds: {redis_client.get('session')}")

if __name__ == "__main__":
    main()
```

- python app
- 4 example
  - basic operation set and get
  - list
  - hashes
  - session expiration

Ca' Foscari University of Venice

Redis with docker

```
Starting docker_redis_1 ... done
Starting docker_redis-app_1 ... done
Attaching to docker_redis_1, docker_redis-app_1
redis_1      | 1:C 07 Nov 2024 23:39:27.354 # WARNING Memory overcommit must be enabled! Without it, a background save o
r replication may fail under low memory condition. Being disabled, it can also cause failures without low memory conditi
on, see https://github.com/jemalloc/jemalloc/issues/1328. To fix this issue add 'vm.overcommit_memory = 1' to /etc/sysct
l.conf and then reboot or run the command 'sysctl vm.overcommit_memory=1' for this to take effect.
redis_1      | 1:C 07 Nov 2024 23:39:27.355 * oO0OoO0OoO0Oo Redis is starting oO0OoO0OoO0Oo
redis_1      | 1:C 07 Nov 2024 23:39:27.355 * Redis version=7.4.1, bits=64, commit=00000000, modified=0, pid=1, just sta
rted
redis_1      | 1:C 07 Nov 2024 23:39:27.355 * Configuration loaded
redis_1      | 1:M 07 Nov 2024 23:39:27.355 * monotonic clock: POSIX clock_gettime
redis_1      | 1:M 07 Nov 2024 23:39:27.357 * Running mode=standalone, port=6379.
redis_1      | 1:M 07 Nov 2024 23:39:27.357 * Server initialized
redis_1      | 1:M 07 Nov 2024 23:39:27.358 * Reading RDB base file on AOF loading...
redis_1      | 1:M 07 Nov 2024 23:39:27.358 * Loading RDB produced by version 7.4.1
redis_1      | 1:M 07 Nov 2024 23:39:27.358 * RDB age 898 seconds
redis_1      | 1:M 07 Nov 2024 23:39:27.358 * RDB memory usage when created 0.90 Mb
redis_1      | 1:M 07 Nov 2024 23:39:27.358 * RDB is base AOF
redis_1      | 1:M 07 Nov 2024 23:39:27.358 * Done loading RDB, keys loaded: 0, keys expired: 0.
redis_1      | 1:M 07 Nov 2024 23:39:27.359 * DB loaded from base file appendonly.aof.1.base.rdb: 0.001 seconds
redis_1      | 1:M 07 Nov 2024 23:39:27.359 * DB loaded from incr file appendonly.aof.1.incr.aof: 0.001 seconds
redis_1      | 1:M 07 Nov 2024 23:39:27.359 * DB loaded from append only file: 0.002 seconds
redis_1      | 1:M 07 Nov 2024 23:39:27.359 * Opening AOF incr file appendonly.aof.1.incr.aof on server start
redis_1      | 1:M 07 Nov 2024 23:39:27.359 * Ready to accept connections tcp
redis-app_1  | === Example 1: Basic operations ===
redis-app_1  | Retrieved value: Redis with Docker
redis-app_1  |
redis-app_1  | === Example 2: Lists ===
redis-app_1  | Students list: ['Mark', 'Anna', 'Louis', 'Mark', 'Anna', 'Louis']
redis-app_1  |
redis-app_1  | === Example 3: Hashes ===
redis-app_1  | User data: {'name': 'John', 'surname': 'Smith', 'age': '25'}
redis-app_1  |
redis-app_1  | === Example 4: Key expiration ===
redis-app_1  | Key 'session' set with 10 seconds expiration
redis-app_1  | Current value: active
redis-app_1  | Value after 11 seconds: None
docker_redis-app_1 exited with code 0
```

Ca' Foscari University of Venice

Redis with docker

- Execution flow:
  - Docker Compose creates a virtual network
  - Starts the "redis" container
  - Waits for it to be ready
  - Starts the "redis-app" container
  - Applications communicate through Docker's internal network

```
[redis-app container] ---> [redis container]
     (client)                  (server)
```

- Important notes:
  - Containers are isolated but can communicate with each other
  - "redis" is accessible from "redis-app" using just the service name as hostname
  - Redis data is persistent thanks to the redis_data volume
  - Only the Redis port is exposed to the host, the Python app is not

- Key concepts:
  - Container isolation
  - Inter-container networking
  - Data persistence
  - Service discovery